



CST3170

Artificial Intelligence (AI)

Machine Learning Coursework

Date of Submission: 12/12/25

Student Name: Koller Melanie Turinabo Busingo

Student ID Number: M00972547

Lab Tutor: Dr Suraj Juddoo

Table of Contents

Introduction.....	3
<i>k</i>-Nearest Neighbor: Evaluation of Baseline Model.....	3
Algorithm Description.....	3
Algorithm Justification.....	4
Two-Fold Cross Validation Procedure	4
Comparison of Confusion Matrices: Euclidean vs Manhattan Distance.....	4
Analysis.....	6
Support Vector Machine (SVM): Evaluation of Model.....	6
Algorithm Description.....	6
Algorithm Justification.....	7
Kernel Variants Evaluated	7
Two-Fold Cross Validation Procedure	8
Confusion matrix	8
Analysis.....	9
Extension – MLP Implementation Attempt.....	10
Algorithm Description.....	10
Comparison of All Algorithms	10
Conclusion.....	12
References.....	13
Self-marking sheet.....	13

Table of Figures

FIG.1:COMPARISON OF K-NN ACCURACY ACROSS DIFFERENT DISTANCE METRICS FOR BOTH FOLDS.....	4
FIG.2: CONFUSION MATRIX FOR BEST EUCLIDEAN CONFIGURATION SHOWING CORRECT CLASSIFICATIONS AND THE MOST FREQUENT MISCLASSIFICATIONS.....	5
FIG.3: CONFUSION MATRIX FOR BEST MANHATTAN CONFIGURATION SHOWING CORRECT CLASSIFICATIONS AND THE MOST FREQUENT MISCLASSIFICATIONS.....	6
FIG.4: TWO-FOLD AVERAGE ACCURACY FOR EACH SVM KERNEL CONFIGURATION, ILLUSTRATING OPTIMAL HYPERPARAMETER SELECTION.	7
FIG. 5: PERFORMANCE COMPARISON OF SVM KERNEL CONFIGURATION ACROSS BOTH FOLDS.....	8
FIG.6: CONFUSION MATRIX FOR THE BEST PERFORMING SVM CONFIGURATION	9

Introduction

This report presents the implementation and evaluation of machine learning algorithms to classify handwritten digits using the UCI Optical Recognition of Handwritten Digits dataset. Two classifiers were implemented from scratch in Java: k-Nearest Neighbors (k-NN), Support Vector Machine (SVM), as well as an attempt at Multi-Layer Perceptron (MLP). Each model was evaluated using a two-fold cross-validation, where each dataset alternately serves as the training and test set.

The task involves classifying 8×8 pixel images of digits (0–9). The datasets were preprocessed according to the method described by NIST (1998), in which 32×32 bitmaps are divided into non-overlapping 4×4 blocks, and the number of black pixels in each block is counted, producing an 8×8 input matrix with integer values from 0 to 16. This reduces dimensionality while maintaining invariance to small distortions. The preprocessing ensures the data is normalized and ready for instance-based and parametric classifiers.

The evaluation focuses on accuracy as the primary metric and uses confusion matrices to identify specific misclassifications among digit classes. The results are analyzed in terms of hyperparameter selection, distance metrics, and overall classifier performance.

k-Nearest Neighbor: Evaluation of Baseline Model

Algorithm Description

The k-Nearest Neighbors (k-NN) algorithm is an instance-based (lazy) learning method that classifies a new data point based on the labels of its k closest neighbors in the feature space. It operates under the assumption that similar data points cluster together, meaning a sample is likely to belong to the same class as its nearest neighbors (Halder et al., 2024).

The Euclidean distance and Manhattan distance metrics were tested to evaluate their effect on classification performance:

- Euclidean distance: $d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$
- Manhattan distance: $d(x, y) = \sum_{i=1}^n |x_i - y_i|$

Odd k values (1, 3, 5, 7) were used to avoid voting ties (Ito, Meenakshi & Singh, 2021). Then the optimal k was selected based on the average two-fold accuracy.

Algorithm Justification

k-NN was chosen for several advantages in handwritten digit classification. Being non-parametric, it does not make assumptions about the underlying data distribution. Halder (2024) emphasizes that it naturally supports multi-class classification, making it suitable for the 10 digit classes. Additionally, the algorithm is interpretable, as predictions are based directly on actual examples from the training dataset. Finally, k-NN has proven performance, with a well-established UCI baseline of 98.25% accuracy (NIST, 1998). The Euclidean distance metric was preferred over Manhattan because it better captures spatial relationships in pixel space and is sensitive to magnitude differences, which is important for normalized digit data (Lubis, 2020). Empirical results confirmed that Euclidean slightly outperforms Manhattan for this dataset.

Two-Fold Cross Validation Procedure

A two-fold cross-validation was performed. In Fold 1, dataset1.csv was used for training and dataset2.csv for testing, while in Fold 2, the roles of the datasets were reversed. Accuracy and confusion matrices were recorded for each fold, and hyperparameters tested included k values of 1, 3, 5, and 7, along with both Euclidean and Manhattan distance metrics.

Distance Metric	k	Fold 1 Accuracy (%)	Fold 2 Accuracy (%)	Average Accuracy (%)	Baseline Accuracy (%)	Status
Euclidean	1	98.04	98.47	98.2562	98.25	Surpassed
Manhattan	1	97.30	97.94	97.6157	98.25	Below

Fig.1: Comparison of k-NN accuracy across different distance metrics for both folds

Comparison of Confusion Matrices: Euclidean vs Manhattan Distance

Euclidean Distance (k = 1)

This configuration achieved an average accuracy of **98.2562%**, slightly surpassing the UCI baseline. Both folds show a strong diagonal structure, indicating highly consistent correct predictions across all digit classes.

Key Misclassification Patterns:

- **1 ↔ 8:** 18 instances

- **9 → 4:** 7 instances
- **7 ↔ 9:** 5 instances
- **3 ↔ 9:** 4 instances

These errors mainly occur between digits with visually overlapping stroke patterns (e.g., curved top of 9 vs. open loop of 4). Importantly, the patterns remained stable across both folds, reflecting good robustness despite small differences in dataset composition.

Confusion Matrix:											
A\P	0	1	2	3	4	5	6	7	8	9	Total
0	268	0	0	0	0	0	1	0	0	0	269
1	0	288	0	0	0	0	0	1	0	1	290
2	0	0	280	0	0	0	0	0	0	0	280
3	0	1	0	269	0	1	0	0	1	3	275
4	0	1	0	0	285	0	2	1	0	4	293
5	0	1	0	1	0	276	0	0	2	5	285
6	0	2	0	0	0	0	272	0	0	0	274
7	0	0	0	0	0	0	0	292	0	0	292
8	0	6	0	0	0	0	0	0	260	0	266
9	0	2	0	1	2	1	0	3	0	277	286

Fig.2: Confusion matrix for best Euclidean configuration showing correct classifications and the most frequent misclassifications

Manhattan Distance (k = 1)

The Manhattan variant achieved an average accuracy of **97.6157%**, falling **0.6343% below** the UCI baseline and consistently below the Euclidean metric across both folds.

Key Misclassification Patterns:

- **8 misclassified as 1:** frequent in both folds
- **9 ↔ 4:** remains a recurring confusion point
- **7 misclassified as 9 or 3:** increased compared to Euclidean
- Slightly more scattered off-diagonal errors in digits 1, 3, 8, and 9

Compared to Euclidean, Manhattan introduces more variability and less compact class separation, which explains the broader spread of errors and lower overall accuracy.

Confusion Matrix(Actual vs Predicted):											
A\P	0	1	2	3	4	5	6	7	8	9	Total
0	268	0	0	0	0	0	1	0	0	0	269
1	0	288	0	0	0	0	0	1	0	1	290
2	0	1	279	0	0	0	0	0	0	0	280
3	0	3	1	266	0	1	0	0	2	2	275
4	0	2	0	0	284	0	1	1	0	5	293
5	0	1	0	1	0	275	0	0	2	6	285
6	0	2	0	0	0	0	272	0	0	0	274
7	0	0	0	0	0	0	0	292	0	0	292
8	0	9	0	1	0	1	0	0	253	2	266
9	0	2	0	1	4	0	0	3	1	275	286

Fig.3: Confusion matrix for best Manhattan configuration showing correct classifications and the most frequent misclassifications

Analysis

Euclidean distance outperformed Manhattan due to its ability to capture subtle pixel intensity patterns effectively (Halder et al., 2024). Slight differences between folds suggest minor dataset imbalance or distribution differences.

Halder et al. (2024), also highlight limitations of k-NN including its $O(n^2)$ computational complexity during classification and the need to store the entire training set, making it memory intensive. Furthermore, performance is sensitive to the choice of k. Future improvements could include optimising the choice of k through finer-grained cross-validation, experimenting with alternative distance metrics (such as Minkowski), and applying dimensionality-reduction techniques like PCA to reduce noise and improve nearest-neighbour comparisons.

In summary, these results demonstrate that even a simple instance-based method can achieve near-optimal performance on well-preprocessed handwritten digit datasets.

Support Vector Machine (SVM): Evaluation of Model

Algorithm Description

The Support Vector Machine used in this project acts as a maximum-margin classifier that separates handwritten digit classes by identifying the optimal hyperplane. Because handwritten digits are not linearly separable in their original 64-dimensional pixel space, the model relies on kernel functions that implicitly map inputs into a higher-dimensional feature space where linear separation becomes possible (Du et al. 2024).

This non-linear mapping follows the kernel formulation

$$\mathbf{K}(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_j),$$

which enables complex decision boundaries without explicitly computing $\phi(\mathbf{x})$ (Du et al. 2024).

A one-vs-rest multi-class strategy was adopted because standard SVMs are binary classifiers. This approach trains one classifier per digit, distinguishing that digit from all others. It is widely used in handwritten recognition tasks due to its simplicity, stability, and strong empirical performance.

Training was performed using the Sequential Minimal Optimization (SMO) procedure, which breaks the large quadratic optimisation problem into analytically solvable sub-problems. SMO was selected because

it is computationally efficient, avoids the need for a full matrix solver, and is well-suited for medium-sized datasets such as this one (Du et al. 2024).

Algorithm Justification

Roman et al. (2021), emphasises that SVM performance depends heavily on correct kernel choice and hyperparameter values—especially the regularisation parameter C and kernel-specific parameters such as gamma, γ in the RBF kernel.

This algorithm is well suited to the dataset because it can capture complex pixel-level patterns, apply the maximum-margin principle to improve generalisation, and leverage the RBF kernel’s ability to model local smoothness in digit images. Furthermore, Roman et al. (2021) highlight that for datasets like the UCI handwritten digits, the RBF kernel consistently performs comparably to more complex composite kernels, suggesting that modelling smoothness properties alone is often sufficient for highly accurate classification.

Kernel Variants Evaluated

Four kernel functions were evaluated. The linear kernel served as a baseline but was expected to perform poorly due to the non-linear distribution of pixel intensities, which aligns with the findings of Roman et al. (2021). The RBF kernel was tested because it captures local similarity by controlling the radius of influence through its γ parameter. A polynomial kernel of degree 2 and 3 was also evaluated, as it provides curved decision boundaries; however, Roman et al. (2021) note its rigidity, and Du et al. (2024) highlight that polynomial kernels become problematic with large sample sizes because the representation lacks uniqueness under certain conditions. Finally, the sigmoid kernel, inspired by neural network activation functions, was included for comparison.

Finding Optimal Configuration (Two-fold Average)			
Kernel: linear	, C=1.0	, Fold1: 9.86%, Fold2: 10.18%, Avg: 10.02%	← NEW BEST
Kernel: linear	, C=10.0	, Fold1: 9.86%, Fold2: 10.18%, Avg: 10.02%	
Kernel: sigmoid	, C=1.0	, Fold1: 40.25%, Fold2: 57.47%, Avg: 48.86%	← NEW BEST
Kernel: sigmoid	, C=10.0	, Fold1: 38.93%, Fold2: 57.26%, Avg: 48.10%	
Kernel: sigmoid	, C=100.0	, Fold1: 39.22%, Fold2: 57.33%, Avg: 48.27%	
Kernel: rbf	, C=1.0, gamma=0.010	, Fold1: 98.15%, Fold2: 98.40%, Avg: 98.27%	← NEW BEST
Kernel: rbf	, C=10.0, gamma=0.001	, Fold1: 9.79%, Fold2: 10.39%, Avg: 10.09%	
Kernel: rbf	, C=1.0, gamma=0.015	, Fold1: 98.08%, Fold2: 98.54%, Avg: 98.31%	← NEW BEST
Kernel: rbf	, C=1.0, gamma=0.020	, Fold1: 98.26%, Fold2: 98.58%, Avg: 98.42%	← NEW BEST ✓
Kernel: poly	, C=1.0, degree=2	, Fold1: 16.41%, Fold2: 12.78%, Avg: 14.59%	← NEW BEST
Kernel: poly	, C=10.0, degree=3	, Fold1: 10.14%, Fold2: 9.57%, Avg: 9.86%	

Fig.4: Two-fold average accuracy for each SVM kernel configuration, illustrating optimal hyperparameter selection.

Two-Fold Cross Validation Procedure

The same two-fold cross-validation procedure used for k-NN was applied here, with each dataset serving alternately as training and test set. This approach helps assess whether any dataset-specific bias affects accuracy.

Multiple hyperparameters were evaluated. C values of 1.0, 10.0, and 100.0 were selected to test different levels of regularisation, as the C parameter controls the trade-off between margin width and classification error. Roman et al. (2021) identify C as a major performance factor, and Du et al. (2024) confirm that different C strengths can be beneficial when class distributions vary. The RBF γ values of 0.010, 0.015, and 0.020 were chosen to assess how tightly the model fits local pixel patterns, with smaller γ producing smoother boundaries and larger γ capturing finer detail. Polynomial degrees 2 and 3 were selected to test low-order curved boundaries without excessive model complexity.

After evaluating all configurations, the optimal model was the RBF kernel with $C = 1.0$ and $\gamma = 0.020$, which achieved the highest two-fold average accuracy of 98.42%, surpassing the upper bound of the UCI SVM baseline (98.275%).

Algorithm	Configuration	Fold 1 Accuracy (%)	Fold 2 Accuracy (%)	Average Accuracy (%)	Status
Linear SVM	$C=1.0$	9.86	10.18	10.02	Below
Sigmoid SVM	$C=1.0$	40.25	57.47	48.86	Below
Polynomial SVM	$C=1.0, \text{deg}=2$	16.41	12.78	14.59	Below
Rbf SVM	$C=1.0, \gamma=0.020$	98.26	98.58	98.42	Above

Fig. 5: Performance comparison of SVM kernel configuration across both folds

Confusion matrix

The best-performing configuration (Fold 2: Train on dataset2 and Test on dataset1) demonstrates high accuracy across all digit classes. Most digits were classified correctly, with only a small number of confusions. Misclassifications primarily occurred between 4, 5, and 9, and occasionally between 1 and 7,

which share visually similar stroke patterns in 8×8 resolution.

Confusion Matrix:											
A \ P	0	1	2	3	4	5	6	7	8	9	Total
0	268	0	0	0	0	0	1	0	0	0	269
1	0	288	0	0	0	0	0	1	0	1	290
2	0	0	280	0	0	0	0	0	0	0	280
3	0	1	0	271	0	1	0	0	1	1	275
4	0	1	0	0	285	0	2	1	0	4	293
5	0	1	0	0	0	277	0	0	2	5	285
6	0	2	0	0	0	0	272	0	0	0	274
7	0	0	0	0	0	0	0	292	0	0	292
8	0	6	0	0	0	0	0	0	260	0	266
9	0	2	0	1	2	0	0	4	0	277	286

Fig.6: Confusion matrix for the best performing SVM configuration

Analysis

The RBF kernel achieved the highest performance because $\gamma = 0.020$ provided an effective balance between capturing local detail in the pixel structure and maintaining smooth decision boundaries. Both folds produced consistently strong results, with Fold 2 showing slightly higher accuracy (98.58%), suggesting mild asymmetry between the datasets. Small changes in γ produced noticeable performance differences, reinforcing the sensitivity of the RBF kernel noted by Roman et al. (2021). Misclassifications mainly arose from digits with overlapping or ambiguous stroke patterns at low resolution.

Limitations include increased computational cost due to SMO's need to store a large number of support vectors, high sensitivity to hyperparameter selection (especially γ), and the absence of feature extraction or data augmentation, which could otherwise improve robustness.

Potential improvements include incorporating feature extraction methods, applying data augmentation to increase variability, expanding the hyperparameter search through grid search or Bayesian optimisation, and exploring ensemble SVMs that combine multiple γ values to stabilise predictions (Du et al., 2024).

Overall, the SVM model using an RBF kernel with $C = 1.0$ and $\gamma = 0.020$ achieved an average accuracy of 98.42%, exceeding the UCI baseline upper limit of 98.275%. This confirms that, when appropriately tuned, SVMs remain highly effective for handwritten digit recognition tasks, capturing the non-linear structure of the 8×8 digit dataset. Despite higher computational cost compared to simpler algorithms, the SVM delivered the strongest performance of all evaluated models.

Extension – MLP Implementation Attempt

Algorithm Description

A Multi-Layer Perceptron neural network was implemented for handwritten digit classification, featuring a feedforward architecture with backpropagation training. The network utilized sigmoid activation functions in hidden layers for non-linearity and softmax activation in the output layer for multi-class probability distribution. Xavier/Glorot initialization was applied to weights for training stability.

Despite correct architectural design principles, the MLP achieved only 9.86% accuracy equivalent to random guessing. Critical issues included numerical instability causing training loss to become NaN after the first epoch, suspected backpropagation gradient errors, and implementation completed late in the development cycle with limited debugging time.

The MLP implementation provided valuable insights into neural network development challenges:

- Neural networks require meticulous attention to numerical stability
- Backpropagation implementation is error-prone without gradient checking
- Algorithm complexity should match available development time
- k-NN and SVM proved more reliable and effective within this project's timeframe

Comparison of All Algorithms

To compare performance across models, both k-NN and SVM were evaluated under the same two-fold cross-validation setup used earlier. Each algorithm was trained on one dataset and tested on the other, then the roles were reversed to measure generalisation across distributions.

Accuracy Comparison Across All Algorithms

Algorithm	Distance/Kernel	Fold1 (%)	Fold 2 (%)	Avg Accuracy (%)	Baseline	Comparison
k-NN	Euclidean(k=1)	98.4	98.47	98.26	98.25	Slightly above
k-NN	Manhattan (k=1)	97.30	97.94	97.62	98.25	Below

SVM	RBF(C=1, $\gamma=0.020$)	98.26	98.58	98.42	96.88 - 98.28	Above upper
-----	------------------------------	-------	-------	-------	------------------	----------------

Confusion Matrix Insights

Across all models, the confusion matrices show strong diagonal dominance, meaning most digits are correctly classified. The most consistent misclassifications appear between visually similar digits such as:

- 1 ↔ 8
- 4 ↔ 9
- 7 ↔ 9
- 3 ↔ 9

These pairs share stroke structures or similar silhouette shapes, which explains why both distance-based and margin-based classifiers struggle slightly with them. SVM, however, reduces several of these errors compared to k-NN Manhattan.

Effect of Preprocessing

Normalisation played a key role in improving both k-NN and SVM performance:

- For k-NN, normalisation ensures no single pixel dominates the distance computation, which is especially important when using Euclidean distance.
- For SVM, normalisation helps the RBF kernel scale distances consistently across features, preventing issues caused by high-variance pixel intensities.

This partly explains why k-NN Manhattan performed weaker: the Manhattan metric is more sensitive to absolute differences in individual pixel values.

Overall, Euclidean k-NN ($k=1$) performs strongly because it captures fine-grained spatial similarity, which aligns well with the curved and continuous structures typical of handwritten digits. In contrast, Manhattan k-NN treats each pixel difference independently, making it less effective when small variations across many pixels accumulate into disproportionately large distance penalties—an issue that becomes more pronounced with rounded or irregular digit shapes. The SVM with an RBF kernel achieves the highest consistency because it learns nonlinear decision boundaries, enabling it to separate digit classes more flexibly and handle subtle shape variations that distance-based methods may struggle to capture.

Conclusion

- Summarise of all conclusions for each model
- Personal Evaluation – what ive learnt(importance to selecting the right set of hyperparametres in order for model to yield the most optimal results, despite it involving a lot of trial and error), struggles -- MLP implementation...
- Future improvements could include implementing eg. Further research on techniques to improve gradient descent, Maybe CNNs for more effective spatial feature extraction, Maybe something to do with dataset itself

This report demonstrates the successful implementation of two models that exceeded the UCI baseline. The k-NN model which showed simplicity, interpretability, and reliable performance, with misclassifications primarily occurring between visually similar digits. The SVM model with an RBF kernel achieved the highest average accuracy, effectively capturing non-linear patterns, albeit at a higher computational cost. Lastly, the MLP, although theoretically promising, failed to converge due to numerical instability and limited debugging time, highlighting the practical challenges of implementing neural networks from scratch.

Furthermore, this project underscored the importance of carefully selecting hyperparameters to optimise model performance, and demonstrated that achieving high accuracy often requires iterative testing and fine-tuning. The difficulties encountered with the MLP further reinforced the need for precise gradient descent implementation and attention to numerical stability.

Future improvements to this project could involve exploring advanced neural network architectures, such as Convolutional Neural Networks (CNNs), applying dimensionality reduction or feature extraction techniques, or investigating enhancements to gradient descent. These approaches could enhance model robustness and accuracy, particularly for larger or more complex datasets.

Overall, the project highlighted the value of appropriate algorithm selection, hyperparameter tuning, and preprocessing, while providing practical insights into the challenges of developing neural network models from scratch.

References

Halder, R.K., Uddin, M.N., Uddin, M.A., Aryal, S. and Khraisat, A., 2024. Enhancing K-nearest neighbor algorithm: a comprehensive review and performance analysis of modifications. *Journal of Big Data*, 11(1), p.113.

Itoo, F., Meenakshi and Singh, S., 2021. Comparison and analysis of logistic regression, Naïve Bayes and KNN machine learning algorithms for credit card fraud detection. *International Journal of Information Technology*, 13(4), pp.1503-1511.

Lubis, A.R. and Lubis, M., 2020. Optimization of distance formula in K-Nearest Neighbor method. *Bulletin of Electrical Engineering and Informatics*, 9(1), pp.326-338.

NIST, 1998. Optical Recognition of Handwritten Digits (n.d.) *UC Irvine Machine Learning Repository*. Available at: <https://archive.ics.uci.edu/dataset/80/optical+recognition+of+handwritten+digits> (Accessed: 18th Novemeber).

Roman, I., Santana, R., Mendiburu, A. and Lozano, J.A., 2021. In-depth analysis of SVM kernel learning and its components. *Neural Computing and Applications*, 33(12), pp.6575-6594.

Self-marking sheet

Mark	Component	Notes
5/5	Self-marking sheet	Submitted and completed
10/10	Running code	All algorithms compile and execute successfully. The code correctly loads the datasets, performs two-fold cross-validation, and outputs accuracy metrics and confusion matrices. The MLP components are

		included but were not executed in the main method due to non-optimal performance
5/5	Two fold test	A proper two-fold cross-validation procedure was implemented, with each dataset serving alternately as training and test set. Results are clearly reported per fold and averaged, demonstrating a sound understanding of the evaluation method.
16/20	Quality of code	The code reflects a solid, modular design originally organized into logical packages. It uses descriptive naming, focused functions, and clear comments to ensure readability and maintainability. Named constants are appropriately used, though some literals remain notably in the MLP implementation. Overall, the code is clean, well-structured, and maintainable.
18/20	Report	The report provides a comprehensive introduction with a clear problem statement, detailed algorithm descriptions supported by academic references, and well-presented results including tables and confusion matrices. Insightful comparisons between algorithms are drawn, and limitations and future work are thoughtfully discussed. Minor improvements could include

		additional visualizations and a deeper analysis of the MLP's failure.
15/20	Quality of Results	Strong results were achieved through methodical hyperparameter tuning, with both k-NN and SVM models attaining high accuracy. A clear analytical understanding is demonstrated via detailed confusion matrix analysis, which identifies specific, explainable error patterns. Results are effectively contextualized through comparison to established academic baselines.
15/20	Quality of Algorithm	The algorithms correctly handle the multi-class classification task, and appropriate preprocessing and normalization were applied to prepare the data. A systematic approach to hyperparameter experimentation was taken, testing various configurations of k-values, distance metrics, kernel types, and regularization parameters to optimize performance.